AWS IAM Policy JSON

Definition:
An IAM policy is a JSON document that defines which AWS actions are **allowed or denied** on specific AWS resources..

JSON:
JSON (JavaScript Object Notation) is the structured text format AWS uses to write IAM policies.

Main Policy Elements:

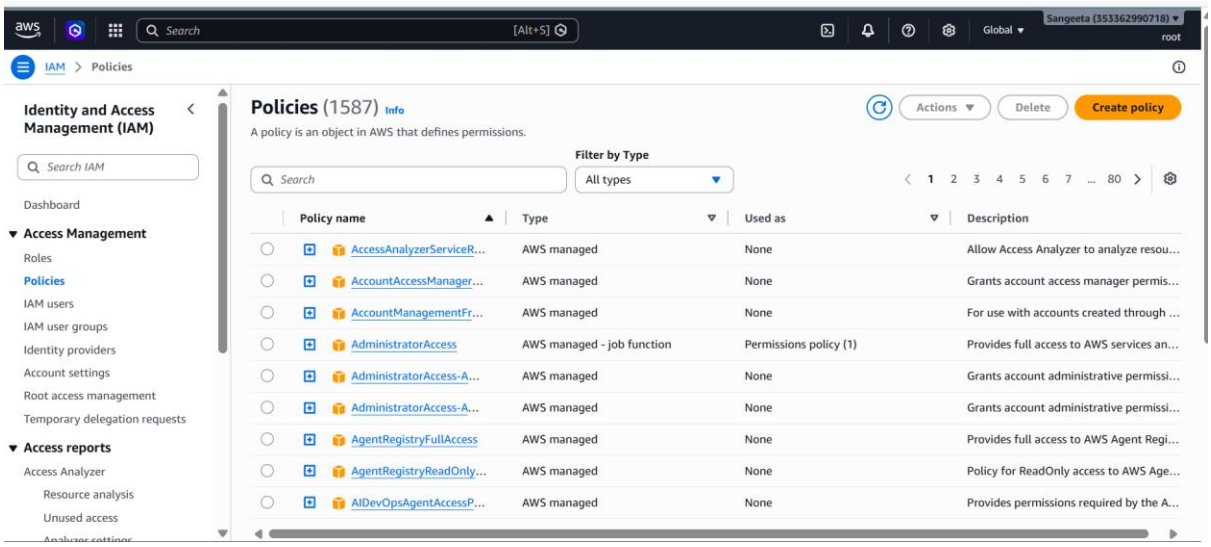
- **Version** – Identifies the IAM policy language version. The standard value is 2012-10-17.
- **Statement** – Contains one or more permission rules.
- **Sid** – Optional name or identifier for a statement.
- **Effect** – Specifies whether access is Allow or Deny.
- **Action** – Specifies the AWS operations that can be performed, such as s3:GetObject.
- **Resource** – Specifies the AWS resource on which the actions can be performed.
- **ARN** – Amazon Resource Name; uniquely identifies an AWS resource.
- **Wildcard (*)** – Represents all applicable actions or resources.
- **Least Privilege** – Gives only the minimum permissions required to perform a task.
- **Implicit Deny** – Access is denied when no policy explicitly allows it.

- **Explicit Deny** – A policy specifically denies an action and overrides an Allow.
- **AccessDenied** – Error displayed when the requested AWS action is not permitted.

Practical: Create and Test a Custom IAM JSON Policy

Step 1: Open IAM Policies

1. Sign in to the AWS Management Console.
2. Open IAM.
3. Go to Policies.
4. Click Create policy.
5. Select the JSON policy editor.



Step 2: Create an S3 Test Bucket

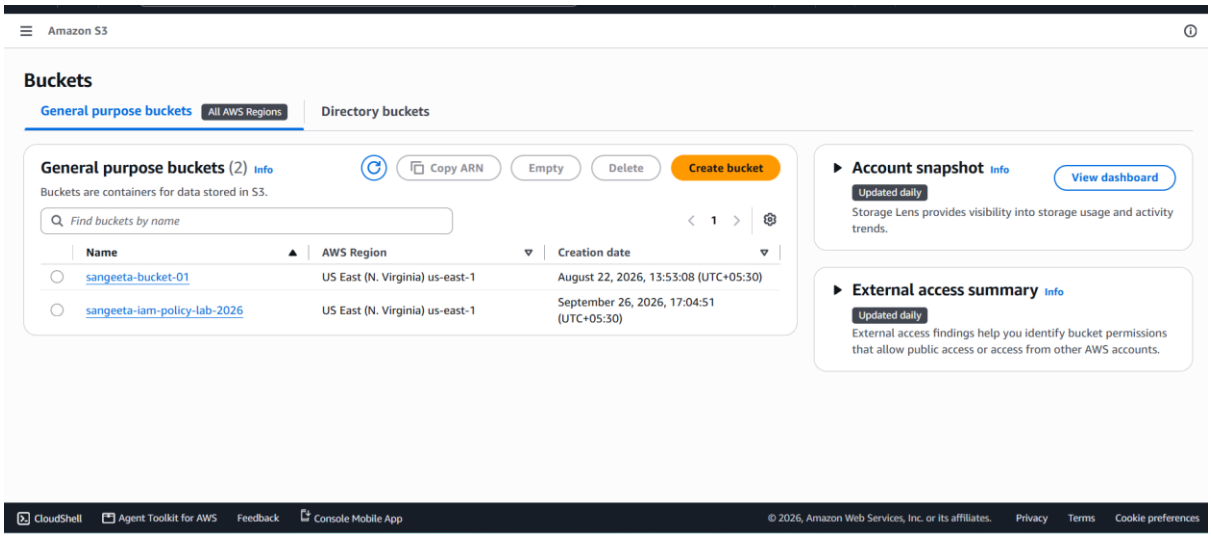
1. Open **Amazon S3**.
2. Click **Create bucket**.

3. Enter the bucket name:

sangeeta-iam-policy-lab-2026

4. Keep **Block all public access** enabled.

5. Create the bucket.



Step 3: Write the IAM Policy JSON :

```
{
 "Version": "2012-10-17",
 "Statement": [
  {
   "Sid": "AllowS3ReadAccess",
   "Effect": "Allow",
   "Action": [
    "s3:ListBucket",
    "s3:GetObject"
   ],
```

```
      "Resource": [
        "arn:aws:s3:::sangeeta-iam-policy-lab-2026",
        "arn:aws:s3:::sangeeta-iam-policy-lab-2026/*"
      ]
    }
  ]
}
```

Policy Explanation:

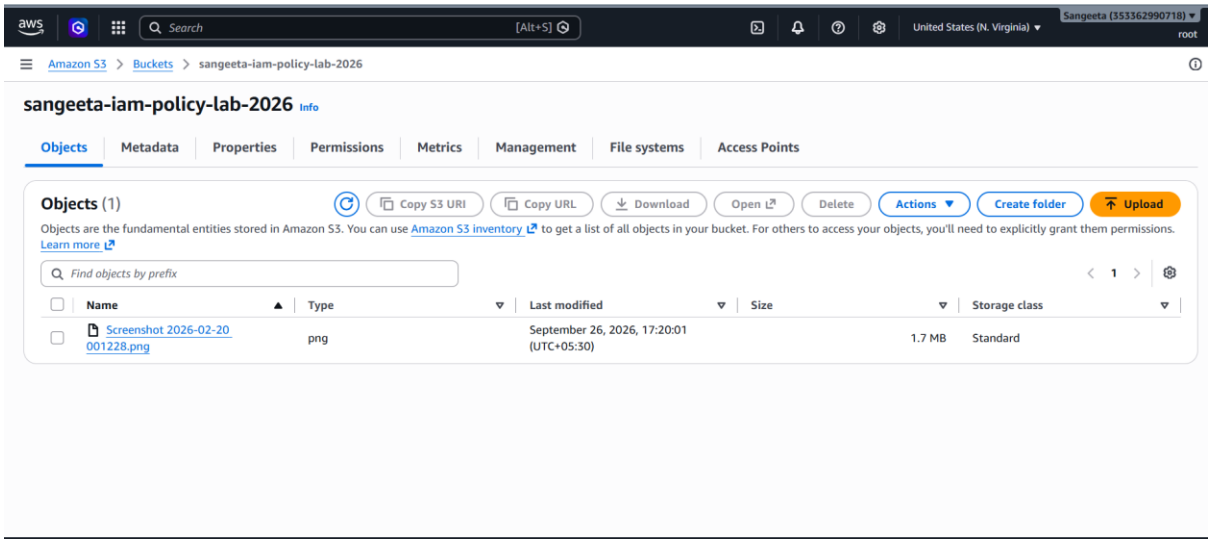
- s3:ListBucket – Allows listing objects inside the specified bucket.
- s3:GetObject – Allows reading/downloading objects.
- Bucket ARN – Identifies the specific S3 bucket.
- /* – Represents objects stored inside the bucket.
- No s3:PutObject permission was provided, so the user cannot upload objects.

Step 4: Create the Custom Policy

1. Entered the policy name:

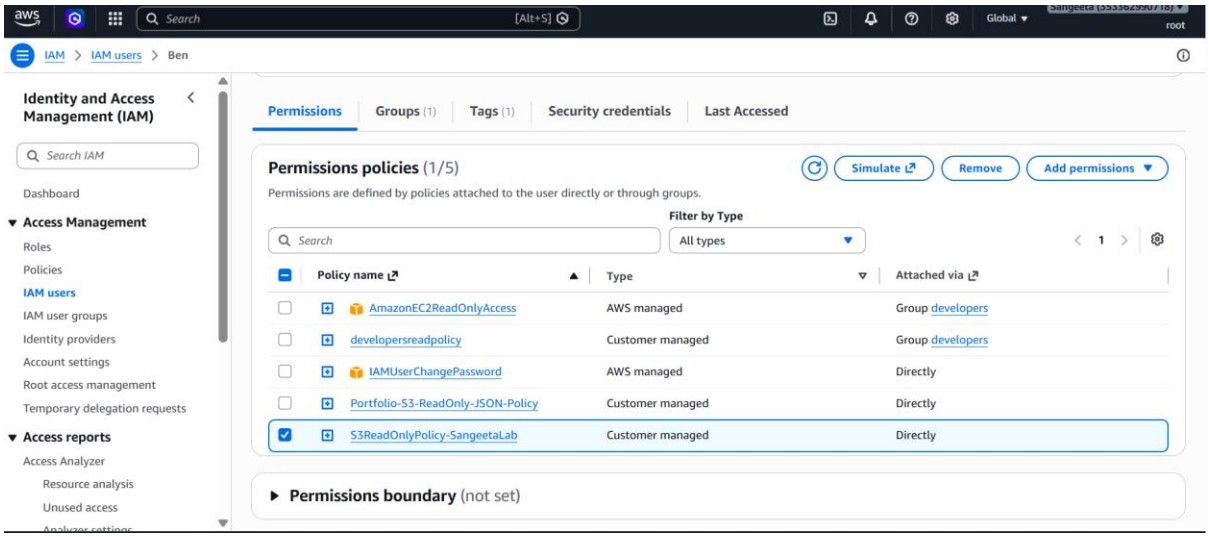
   S3ReadOnlyPolicy-SangeetaLab

2. Added the description for the read-only S3 policy.
3. Created the policy as a Customer managed policy.



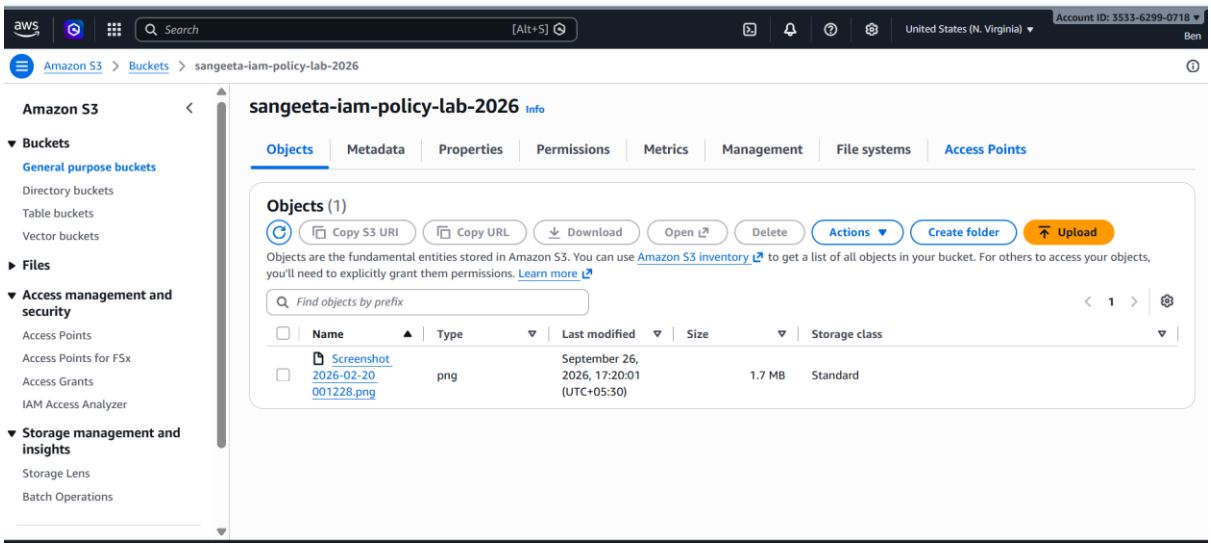
Step 5: Attach the Policy to IAM User Ben

1. Opened IAM → IAM users → Ben.
2. Selected Add permissions.
3. Selected Attach policies directly.
4. Selected S3ReadOnlyPolicy-SangeetaLab.
5. Added the permission to Ben.



Test Object as Administrator :

1. Opened the S3 bucket sangeeta-iam-policy-lab-2026.
2. Uploaded a test .txt file.
3. The object was used to test Ben's read permissions.



Test Allowed Permissions as Ben

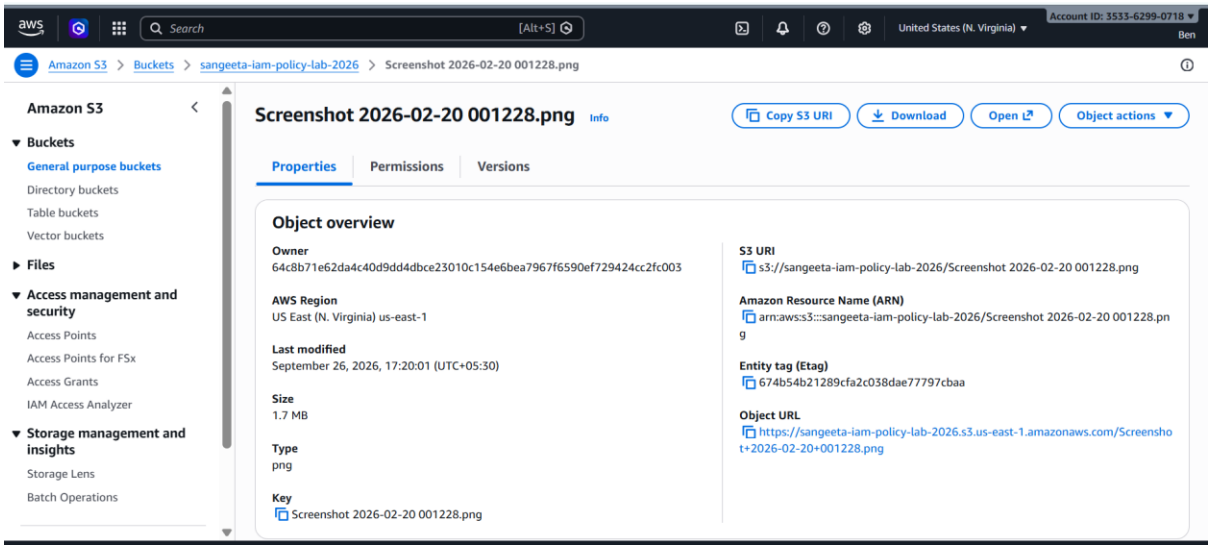
Write:

1. Signed in as IAM user **Ben**.
2. Opened Amazon S3.
3. Opened sangeeta-iam-policy-lab-2026.
4. Ben successfully viewed the objects in the bucket.

Result: s3:ListBucket → **Allowed** ✅

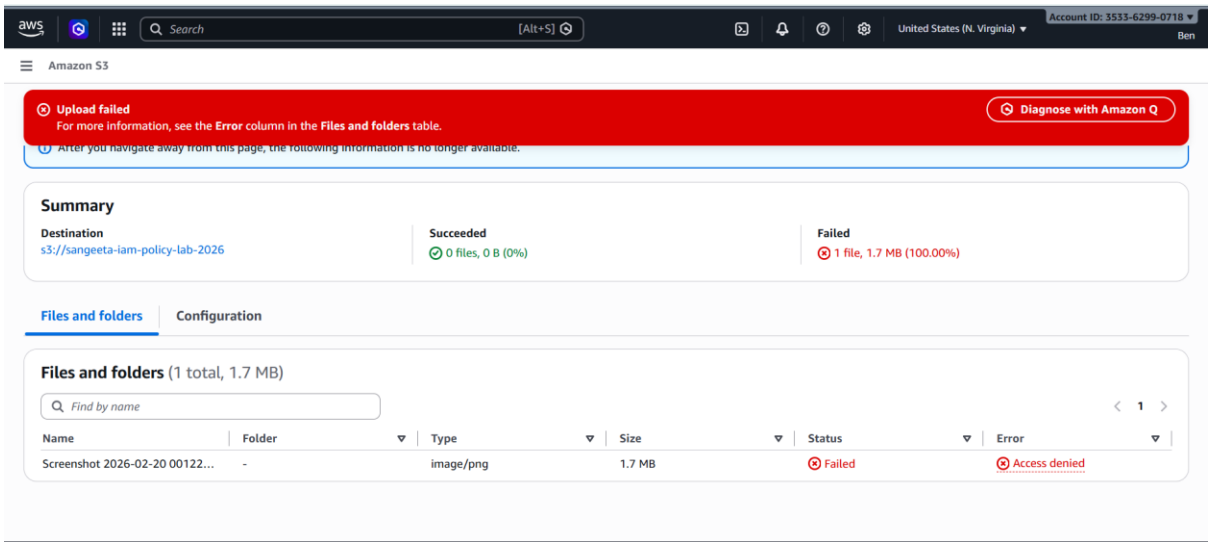
5. Opened/downloaded the test object.

Result: s3:GetObject → **Allowed** ✅



Test Denied Permission

1. Signed in as IAM user **Ben**.
2. Opened the S3 bucket sangeeta-iam-policy-lab-2026.
3. Attempted to upload a new file to the bucket.
4. The upload failed because Ben was not granted the s3:PutObject permission.



IAM Policy Evaluation – Explicit Allow vs Explicit Deny

Scenario:
IAM user Ben already has S3ReadOnlyPolicy-SangeetaLab, which allows read access to the test S3 bucket.

In this practical, another policy will be created with an Explicit Deny. The test will demonstrate how AWS evaluates Allow and Deny permissions.

Create the Explicit Deny Policy

On **IAM → Policies**:

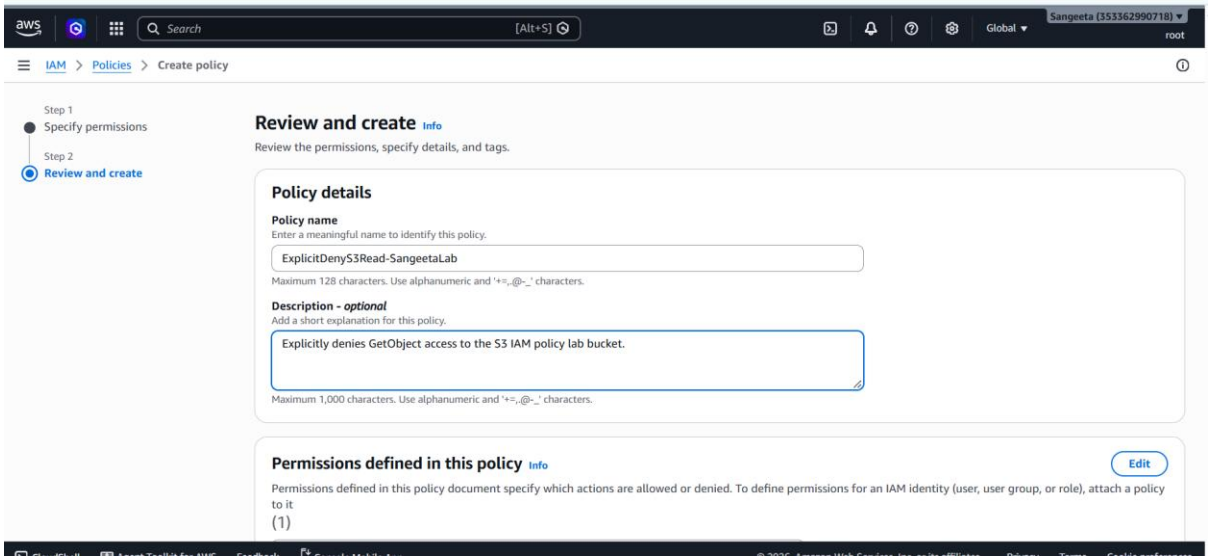
1. Click **Create policy**.
2. Select the **JSON** editor.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "ExplicitDenyS3Read",
      "Effect": "Deny",
      "Action": [
        "s3:GetObject"
      ],
```

```
        "Resource": "arn:aws:s3:::sangeeta-iam-policy-lab-2026/*"
      }
    ]
}
```

Create the Deny Policy

1. Created a customer-managed policy named ExplicitDenyS3Read-SangeetaLab.
2. The policy explicitly denies the s3:GetObject action.
3. This policy will be attached to Ben to test AWS policy evaluation.



Attach Deny Policy to Ben

Stay logged in as root/admin.

1. Go to IAM → IAM users.

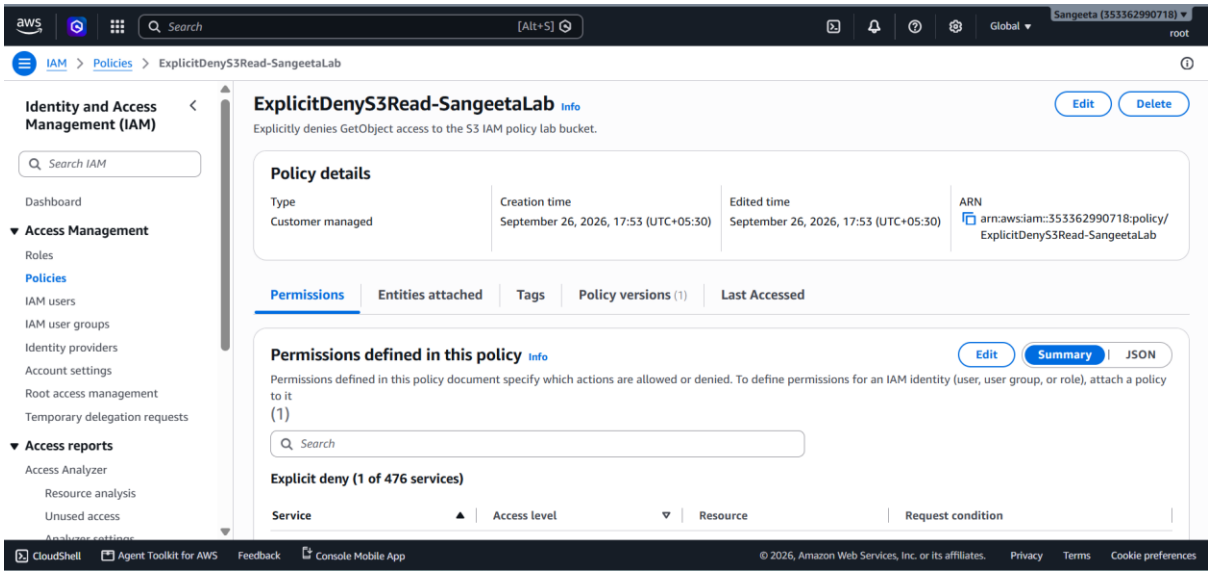
2. Click Ben.
3. Stay on the Permissions tab.
4. Click Add permissions.
5. Choose Attach policies directly.
6. Search for:

ExplicitDenyS3Read-SangeetaLab

7. Select the checkbox.
8. Click Next → Add permissions.

After this, Ben should have both policies:

- S3ReadOnlyPolicy-SangeetaLab → Allows s3:GetObject
- ExplicitDenyS3Read-SangeetaLab → Denies s3:GetObject

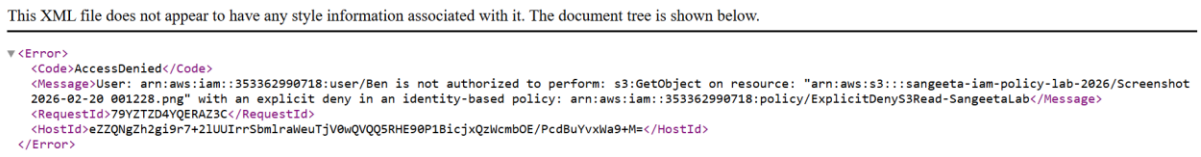


Test Policy Evaluation

1. Signed in as IAM user Ben.

2. Opened the permitted S3 bucket.
3. Ben could still list the objects because s3:ListBucket was allowed.
4. Attempted to download the test object.
5. The download was denied because an Explicit Deny overrides an Allow.

Result:
Allow + Explicit Deny → DENY

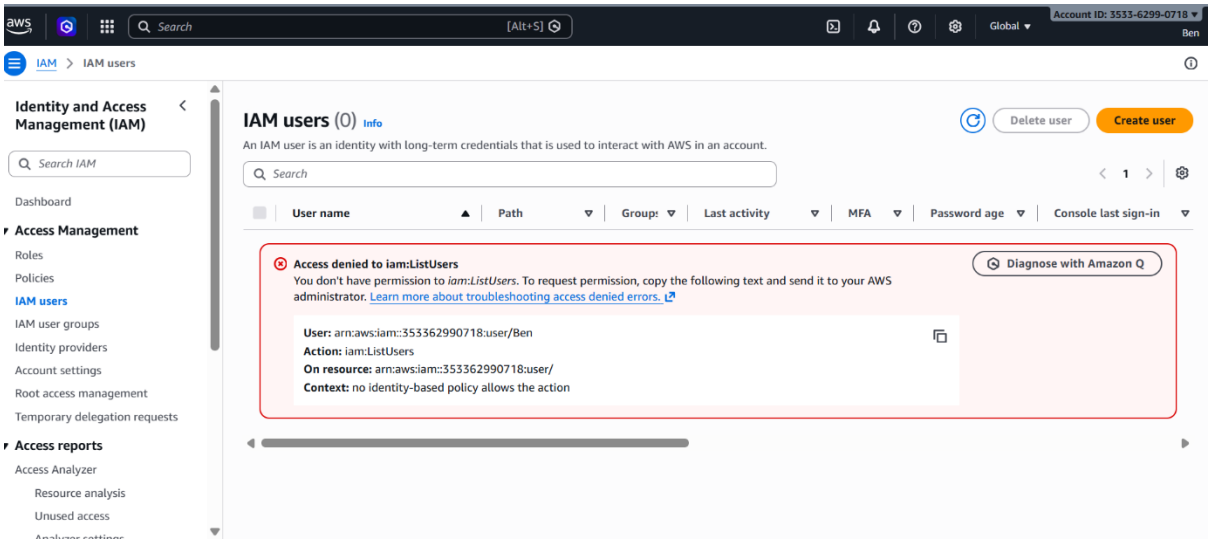

This XML file does not appear to have any style information associated with it. The document tree is shown below.


```
▼<Error>
  <Code>AccessDenied</Code>
  <Message>User: arn:aws:iam::353362990718:user/Ben is not authorized to perform: s3:GetObject on resource: "arn:aws:s3:::sangeeta-iam-policy-lab-2026/Screenshot
  2026-02-20 001228.png" with an explicit deny in an identity-based policy: arn:aws:iam::353362990718:policy/ExplicitDenyS3Read-SangeetaLab</Message>
  <RequestId>79YZTZD4YQERAZ3C</RequestId>
  <HostId>eZZQNgZh2gi9r7+2lUUIrrSbmlraWeuTjV0wQVQQ5RHE90P1BicjxQzWcmbOE/PcdBuYvxWa9+M=</HostId>
</Error>
```


Test Implicit Deny

1. Signed in to AWS as IAM user Ben.
2. Opened IAM → IAM users.
3. Ben attempted to list the IAM users in the AWS account.
4. Ben does not have the iam:ListUsers permission.
5. AWS denied the request because no identity-based policy allows this action.

Result: iam:ListUsers → Implicit Deny



Conditions

Condition:
A Condition in an IAM policy specifies when a policy statement should apply. It allows permissions to be controlled based on specific requirements.

Condition Operator:
Defines how AWS compares the condition value, such as StringEquals, IpAddress, Bool, or DateGreaterThan.

Condition Key:
Specifies the property AWS checks when evaluating the request, such as aws:SecureTransport or aws:SourceIp.

Condition Value:
The value that must match the condition for the policy statement to apply.

Example:

```
"Condition": {
  "Bool": {
    "aws:SecureTransport": "true"
  }
}
```

Practical: Using Conditions in an IAM Policy

Scenario:
A conditional IAM policy will be created to demonstrate how AWS can apply permissions only when specified conditions are satisfied.

Test Resource:
S3 bucket: sangeeta-iam-policy-lab-2026

Create a Conditional IAM Policy

Created an IAM JSON policy that allows S3 read access only when the request uses secure transport

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowS3ReadOnlyWithSecureTransport",
      "Effect": "Allow",
```

```
      "Action": [
        "s3:ListBucket",
        "s3:GetObject"
      ],
      "Resource": [
        "arn:aws:s3:::sangeeta-iam-policy-lab-2026",
        "arn:aws:s3:::sangeeta-iam-policy-lab-2026/*"
      ],
      "Condition": {
        "Bool": {
          "aws:SecureTransport": "true"
        }
      }
    }
  ]
}
```

Condition used: aws:SecureTransport

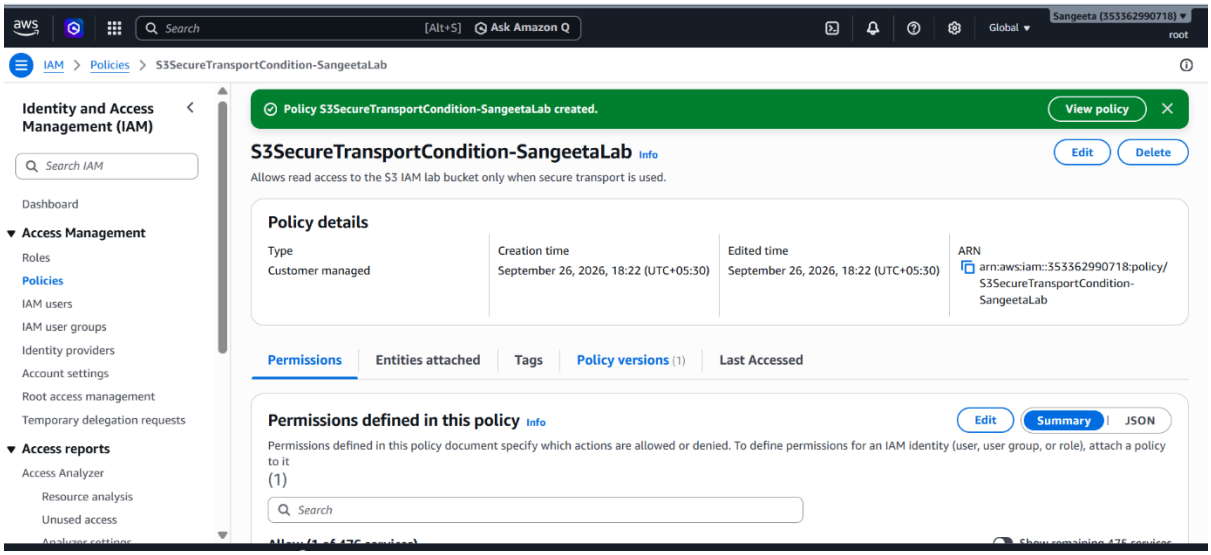
Operator: Bool

Required value: true

Meaning: The Allow statement applies only when the request is made using secure transport.

Attach the Conditional Policy to Ben

1. Opened IAM → IAM users → Ben → Permissions.
2. Attached S3SecureTransportCondition-SangeetaLab.
3. The policy allows S3 read access when the aws:SecureTransport condition is true.



Isolate the Conditional Policy

The previous S3ReadOnlyPolicy-SangeetaLab policy was temporarily detached from Ben so that S3 access could be tested using only the conditional policy.

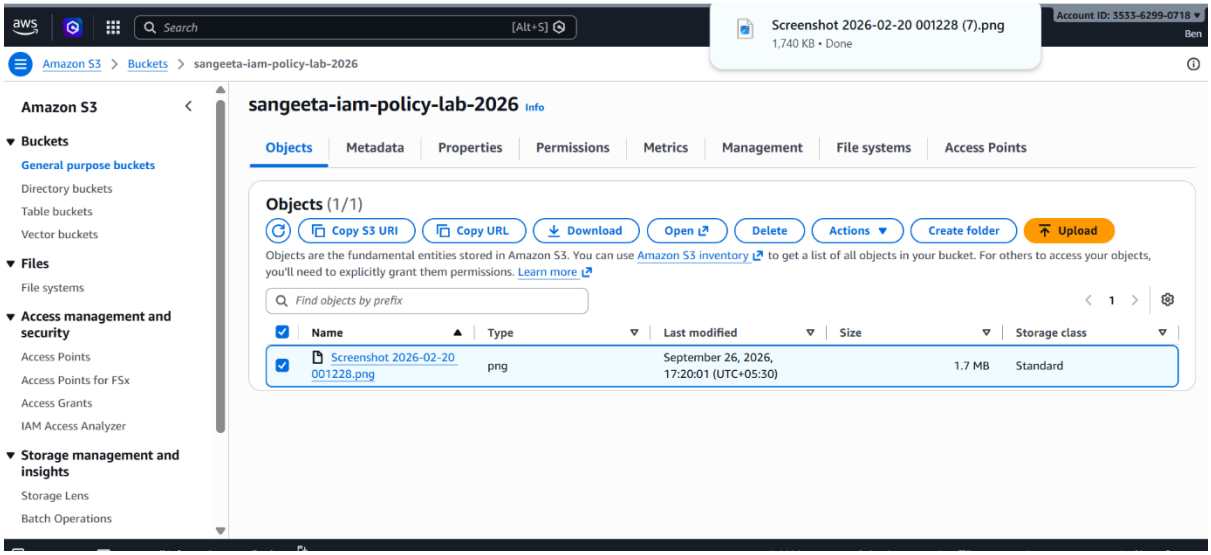
Test the Secure Transport Condition

1. Signed in as IAM user **Ben**.
2. Opened Amazon S3 through the AWS Management Console.
3. Opened the bucket sangeeta-iam-policy-lab-2026.
4. Selected the test object and clicked **Download**.

5. The object downloaded successfully because the request used secure transport (HTTPS).

Result:

aws:SecureTransport = true → **Condition satisfied → ALLOW**



Test When the Condition Is Not Satisfied

For testing purposes, the condition value was temporarily changed from true to false.

Since the AWS Management Console uses HTTPS, the actual request has:

aws:SecureTransport = true

but the policy requires:

aws:SecureTransport = false

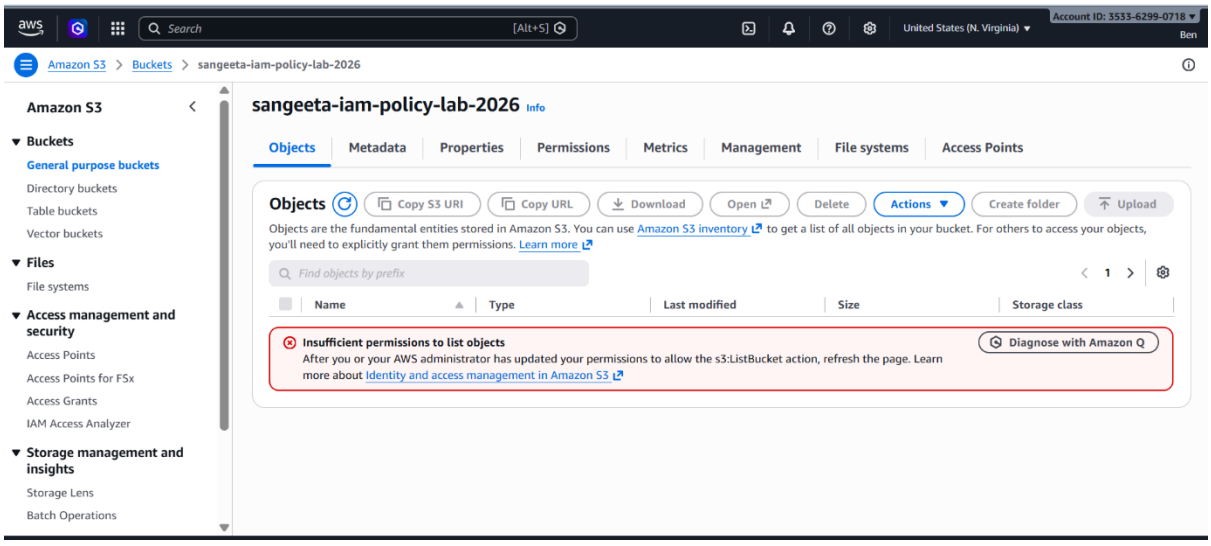
Therefore, the condition should not match and the Allow statement should not apply

Verify the Unsatisfied Condition

1. Signed in as IAM user **Ben**.
2. Accessed Amazon S3 through the AWS Management Console using HTTPS.
3. The policy required aws:SecureTransport = false.
4. The actual request used secure transport (true).
5. Because the condition did not match, the Allow statement did not apply.
6. AWS denied the requested S3 access.

Result:

Required = false + Actual = true → **Condition not satisfied → DENY**



Resource Policy:

A resource-based policy is a JSON policy attached directly to an AWS resource rather than to an IAM user, group, or role.

Principal:
Specifies who is allowed or denied access to the resource.

Identity-Based Policy:
A policy attached to an IAM identity such as a user, group, or role.

Resource-Based Policy:
A policy attached directly to a resource, such as an S3 bucket policy.

S3 Bucket Policy:
A resource-based policy attached directly to an S3 bucket to control access to that bucket and its objects.

Key difference:

Identity policy: *What can Ben do?*
Resource policy: *Who can access this S3 bucket?*

Practical we'll build

We'll reuse:

Ben → sangeeta-iam-policy-lab-2026

But this time we'll control access using an **S3 Bucket Policy** instead of only attaching permissions to Ben.